

Day 7 — Reusable Components

Your simple, friendly guide for today

COMING FROM	KnockoutJS / .NET (C#) → moving to React + TypeScript
TODAY'S GOAL	Build a small, typed UI kit (Button, Input, Table, Modal, Loader, Pagination) and custom hooks — write once, reuse everywhere.

How to use this: Read **Section 1 (Learning Plan)** for each idea with a small example. Then read **Section 2 (Detailed Notes)** slowly, top to bottom — one read should make it click. Keep **Section 3 (Common Mistakes)** handy while you practise, and copy **Section 4 (Concept Notes)** by pen for recall & interviews.

1 Learning Plan

Today you build a small **UI kit** — a set of reusable, typed building blocks (Button, Input, Table, Modal, Spinner, Pagination) plus **custom hooks** (little helper functions that share logic). The HR app then reuses these everywhere instead of copy-pasting.

Think of it like this: in .NET you build **shared user controls** (or Blazor components) and drop them on many screens. You also pull common logic into a **helper/service** class. React does the same — components are the controls, custom hooks are the shared logic.

Note: **global state** (Context + reducers) and **performance** (`memo` , `useMemo` , `useCallback`) are **Day 8** — not today. Today we keep every component **presentational** ("dumb"): it just takes props in and shows UI. No global stores, no data-fetching baked into the visual parts.

Part A — What you will build

1. Button

A reusable button that takes a `variant` (like `primary` / `danger`) **and** every normal button prop (`onClick` , `disabled` , `type` ...). The trick is `React.ComponentPropsWithoutRef<'button'>` — it means "give me all the real `<button>` props for free". We then spread `...rest` onto the element and merge `className` so callers can add their own styles.

```
import React from 'react';

type ButtonVariant = 'primary' | 'secondary' | 'danger';

interface ButtonProps extends React.ComponentPropsWithoutRef<'button'> {
  variant?: ButtonVariant;
}

export function Button({ variant = 'primary', className = '', ...rest }: ButtonProps) {
  return <button className={`btn btn-${variant} ${className}`.trim()} { ...rest} />;
}
```

- **.NET / KnockoutJS:** like a shared `<Button>` user control whose extra attributes pass straight through to the underlying HTML button.
- **You'll be able to:** make one button used across the whole app, still typed exactly like a real `<button>` .

2. Input

A labelled text box that can show an **error message** under it. The key new thing is `forwardRef` — it lets a parent (or a form library) reach the real `<input>` DOM node through your component. Day 6's React Hook Form needs this ref to read and focus the field.

```
import React, { forwardRef } from 'react';

interface InputProps extends React.ComponentPropsWithoutRef<'input'> {
  label: string;
  error?: string;
}

export const Input = forwardRef<HTMLInputElement, InputProps>(
  function Input({ label, error, id, ...rest }, ref) {
    return (
      <div className="field">
        <label htmlFor={id}>{label}</label>
        <input id={id} ref={ref} {...rest} />
        {error && <span className="field-error">{error}</span>}
      </div>
    );
  }
);
```

- **.NET / KnockoutJS:** like a labelled input user control; `forwardRef` is like exposing the inner textbox so outside code can focus/read it.
- **You'll be able to:** drop this straight into React Hook Form (`{ ...register('name')}`) and show validation errors.

3. Table (generic `Table<T>`)

This is where **Day 2 generics** pay off. The table doesn't care *what* it shows — employees, departments, anything. You pass `rows: T[]` and a `columns` list that says how to draw each cell. `T` is a placeholder for "whatever row type you give me", so you get full type-safety and autocomplete inside each column.

```

import React from 'react';

interface Column<T> {
  header: string;
  render: (row: T) => React.ReactNode;
}

interface TableProps<T> {
  rows: T[];
  columns: Column<T>[];
  rowKey: (row: T) => string | number;
}

export function Table<T>({ rows, columns, rowKey }: TableProps<T>) {
  return (
    <table>
      <thead>
        <tr>{columns.map((col) => <th key={col.header}>{col.header}</th>)}</tr>
      </thead>
      <tbody>
        {rows.map((row) => (
          <tr key={rowKey(row)}>
            {columns.map((col) => <td key={col.header}>{col.render(row)}</td>)}
          </tr>
        ))}
      </tbody>
    </table>
  );
}

```

- **.NET / KnockoutJS:** like a generic `GridView` / `DataGrid<T>` where you define columns once and bind any list.
- **You'll be able to:** show an employee table today and reuse the exact same component for departments tomorrow — no rewrite.

4. Modal (portal)

A dialog box. `createPortal` (from `react-dom`) renders the modal **into** `document.body`, outside your normal component tree, so it can float above everything without CSS clipping. It takes `isOpen`, `onClose`, and `children`, and closes on overlay click or the **Escape** key.

```

import React, { useEffect } from 'react';
import { createPortal } from 'react-dom';

interface ModalProps {
  isOpen: boolean;
  onClose: () => void;
  children: React.ReactNode;
}

export function Modal({ isOpen, onClose, children }: ModalProps) {
  useEffect(() => {
    const onKey = (e: KeyboardEvent) => {
      if (e.key === 'Escape') onClose();
    };
    document.addEventListener('keydown', onKey);
    return () => document.removeEventListener('keydown', onKey);
  }, [onClose]);

  if (!isOpen) return null;

  return createPortal(
    <div className="overlay" onClick={onClose}>
      <div className="dialog" onClick={e => e.stopPropagation()}>
        {children}
      </div>
    </div>,
    document.body
  );
}

```

- **.NET / KnockoutJS:** like a shared modal/dialog control; the portal is like rendering the popup at the top window level so nothing clips it.
- **You'll be able to:** pop a confirm/edit dialog anywhere, controlled purely by props.

5. Loader / Spinner

A tiny, purely visual "please wait" indicator. It takes an optional `label` and nothing else — no logic. Small, boring, and reused everywhere a screen is loading.

```

interface SpinnerProps {
  label?: string;
}

export function Spinner({ label = 'Loading...' }: SpinnerProps) {
  return (
    <div className="spinner" role="status" aria-live="polite">
      <span className="spinner-circle" />
      <span>{label}</span>
    </div>
  );
}

```

- **.NET / KnockoutJS:** like a shared "busy indicator" user control you show while a call runs.
- **You'll be able to:** show a consistent loading state on every page with one component.

6. Pagination

Prev / Next buttons plus numbered pages. It is fully **prop-driven**: it doesn't fetch or slice data itself — it just gets `page`, `pageCount`, and calls `onChange(newPage)` when clicked. The parent decides what to do with the new page number.

```

interface PaginationProps {
  page: number;
  pageCount: number;
  onChange: (page: number) => void;
}

export function Pagination({ page, pageCount, onChange }: PaginationProps) {
  const pages = Array.from({ length: pageCount }, (_, i) => i + 1);
  return (
    <nav className="pagination">
      <button disabled={page === 1} onClick={() => onChange(page - 1)}>Prev</button>
      {pages.map((p) => (
        <button key={p} aria-current={p === page} onClick={() => onChange(p)}>
          {p}
        </button>
      ))}
      <button disabled={page === pageCount} onClick={() => onChange(page + 1)}>Next</button>
    </nav>
  );
}

```

- **.NET / KnockoutJS:** like a reusable pager control that raises a "page changed" event for the parent to handle.
- **You'll be able to:** add paging to any list by wiring three props.

7. Custom hooks

A **custom hook** is just a function whose name starts with `use` and which can call other hooks (`useState`, `useEffect` ...). It lets you lift shared logic **out** of components so many components reuse it. Three you'll write today: `useToggle` (open/close the Modal), `useDebounce` (wait a bit before searching), and a data hook like `useEmployees` (hides Day 5's `useEffect` + `fetch`).

```
// useToggle - open/close state for the Modal
import { useState, useCallback } from 'react';

export function useToggle(initial = false) {
  const [isOpen, setIsOpen] = useState(initial);
  const open = useCallback(() => setIsOpen(true), []);
  const close = useCallback(() => setIsOpen(false), []);
  const toggle = useCallback(() => setIsOpen((v) => !v), []);
  return { isOpen, open, close, toggle };
}
```

```
// useDebounce - wait `delay` ms after typing stops (great for search boxes)
import { useState, useEffect } from 'react';

export function useDebounce<T>(value: T, delay = 400): T {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);
  }, [value, delay]);
  return debounced;
}
```

```
// useEmployees - hides the fetch/useEffect; components just get {data, loading, error}
import { useState, useEffect } from 'react';

interface Employee {
  id: number;
  name: string;
  department: string;
}

export function useEmployees() {
  const [employees, setEmployees] = useState<Employee[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    let active = true;
    fetch('/api/employees')
      .then((res) => res.json() as Promise<Employee[]>)
      .then((data) => { if (active) setEmployees(data); })
      .catch(() => { if (active) setError('Could not load employees'); })
      .finally(() => { if (active) setLoading(false); });
    return () => { active = false; };
  }, []);

  return { employees, loading, error };
}
```

- **.NET / KnockoutJS:** like extracting repeated logic into a **service/helper** class that many view-models call.
- **You'll be able to:** reuse the same behaviour in many components and keep the components short and clean.

Putting the kit together

Here the hook drives the Modal and the Button — this is the whole point of the day: small pieces snapping together.


```
import { Button } from './components/ui/Button/Button';
import { Modal } from './components/ui/Modal/Modal';
import { useToggle } from './hooks/useToggle';

export function EmployeePanel() {
  const { isOpen, open, close } = useToggle();
  return (
    <
      <Button variant="primary" onClick={open}>Add employee</Button>
      <Modal isOpen={isOpen} onClose={close}>
        <h2>New employee</h2>
        <Button variant="secondary" onClick={close}>Cancel</Button>
      </Modal>
    </>
  );
}
```

Part B — Extra topics (folded in)

B1. Component API design — props + children + sensible defaults

Design components like a good function: clear inputs (props), clear output (UI). Prefer **composition** — let callers pass `children` instead of hard-coding the inside. Give **sensible defaults** (`variant = 'primary'`) so simple use stays simple. Keep visual components **presentational**: they take data and handlers as props and never fetch or own app-wide state. That comes on Day 8.

- **.NET contrast:** same idea as a well-designed user control — public properties in, events out, no hidden reach into the rest of the app.

B2. Extending native element props + forwardRef (why libraries need the ref)

`React.ComponentPropsWithoutRef<'button'>` gives your component every real prop of that HTML element, so users aren't surprised when `onClick` or `aria-*` "just works". A **ref** is a direct handle to the underlying DOM node. Some tools (focus management, React Hook Form, scroll-into-view) must touch the real element — `forwardRef` passes the ref through your wrapper to that element. Use `ComponentPropsWithRef<'input'>` when the element type also needs to carry a ref.

- **.NET contrast:** exposing the inner control so external code can call `.Focus()` on it.

B3. Rules of Hooks (recap for custom hooks)

Hooks must be called **at the top level** of a component or another hook — never inside `if`, loops, or nested functions — so React can match them up on every render. A **custom hook** must start with `use`,

may call other hooks inside, and returns whatever you want: values, state, and handler functions. That naming isn't just style — it's how React's lint rules know to check it.

- **.NET contrast:** like a helper method with a naming convention the tooling relies on.

B4. Portals (createPortal) — why a Modal renders outside the tree

Normally a component renders where it sits in the tree, which means parent CSS (`overflow: hidden`, `z-index`, transforms) can clip or trap a popup. `createPortal(node, document.body)` renders the modal's markup at the top of the page instead, while **keeping** its normal React props/state/events. So it behaves like part of your component but floats above everything.

- **.NET contrast:** showing a dialog at the top-level window rather than nested inside a panel.

B5. Folder structure for a UI library + barrel export

Keep the kit tidy: one component per folder under `src/components/ui/...`, each with its own file (and later its styles/tests). An `index.ts` **barrel** re-exports everything so callers import from one place.

```
// src/components/ui/index.ts
export { Button } from './Button/Button';
export { Input } from './Input/Input';
export { Table } from './Table/Table';
export { Modal } from './Modal/Modal';
export { Spinner } from './Spinner/Spinner';
export { Pagination } from './Pagination/Pagination';
```

Then: `import { Button, Modal } from '@components/ui';`

- **.NET contrast:** like a shared class library with a clean public namespace so consumers add one `using`.

B6. Light mention — Storybook

Storybook is a tool that lets you develop and preview each component **in isolation**, outside the full app — you see every variant (primary/danger, error state, empty table) on one page. Great for building a UI kit and sharing it. We won't set it up today; just know it exists and it's the standard way teams grow a component library.

B7. .NET contrast (summary)

- **Reusable component** ~ a shared **user control** / **Blazor component**: public props in, events out, reused across screens.
- **Custom hook** ~ **extracting shared logic into a service/helper**: one place to maintain, many callers.

Detailed Notes — Read Once, Understand Everything

The big idea first: stop copy-pasting

Think back to Day 3–6. Every time you needed a button, a table, or a loading spinner, you probably wrote the markup again. That is copy-paste. Copy-paste feels fast today and hurts tomorrow: fix a bug in one place, and the other ten copies stay broken.

Today we fix that with two tools:

- A **reusable component** = a small building block you write **once** and use **everywhere**. You feed it data through **props** (props = the inputs you pass to a component, like arguments to a method). This is exactly like a **Blazor / WinForms user control** or a shared class in C# — write it once, reuse it across screens.
- A **custom hook** = the same idea, but for **logic** instead of markup. When two components share the same stateful logic (loading data, a toggle, a debounce), you pull that logic out into a reusable function. In C# terms: extracting shared code into a **helper method or a service**.

So: reusable component = shared *UI*. Custom hook = shared *logic*. That is the whole day.

Today we build a tiny **UI kit** (a small library of our own components) — Button, Input, Table, Modal, Loader, Pagination — plus a few custom hooks, and our HR app reuses them.

One note up front: global app state (Context, reducers) and performance tuning (`memo` , `useMemo` , `useCallback`) are **Day 8**. Today we keep every component **presentational** (presentational = it just shows what you give it and reports clicks back; it does not own the app's data).

1. What makes a component reusable

Three simple rules:

1. **It takes props** — data to show, and callbacks to report events (callback = a function you pass in so the component can call *you* back, like an event handler).
2. **It does not hardcode content** — no fixed text like "Save Employee" baked inside. That comes from a prop.
3. **It stays presentational** — it does not fetch data or know about your app. It shows what it is given.

Bad (hardcoded, can be used only once):

```
function SaveButton() {  
    return <button className="btn-primary">Save Employee</button>;  
}
```

Good (prop-driven, works anywhere):

```
interface ActionButtonProps {
  label: string;
  onClick: () => void;
}

function ActionButton({ label, onClick }: ActionButtonProps) {
  return <button className="btn-primary" onClick={onClick}>{label}</button>;
}
```

The second one has no idea it is saving an employee. That is the point — it can save, delete, or export. Reusable = it doesn't know or care about your specific screen.

2. Typing props with an interface, `children`, and default values

In TypeScript you describe a component's props with an **interface** — same interfaces you learned on Day 2. This is your component's "public contract," like the parameter list of a C# method. If a caller forgets a prop or passes the wrong type, the compiler complains.

`children` is a special prop: it is whatever you put **between the tags** when you use the component (`<Card> ... this part ... </Card>`). Its type is `ReactNode` — a built-in type meaning "anything React can render": text, numbers, JSX, a list of elements, or nothing.

You give a prop a **default value** with plain JavaScript default parameters (`variant = 'primary'`).

```
import { ReactNode } from 'react';

interface BadgeProps {
  color?: 'green' | 'red'; // ? = optional
  children: ReactNode;    // whatever is placed inside the tags
}

function Badge({ color = 'green', children }: BadgeProps) {
  return <span className={`badge badge-${color}`}>{children}</span>;
}

// Used like: <Badge color="red">Inactive</Badge>
```

`color?` means optional. Because it is optional, we give it a default (`'green'`) so the code never has to deal with `undefined` .

3. Extending native props (so your Button is a *real* button)

Here is a common trap. You build a `Button`, then later you need `disabled`, then `type="submit"`, then `aria-label`, then `onFocus` ... and you keep adding props one by one. Painful.

A real `<button>` already supports all of those. So **borrow its whole prop list** instead of re-declaring it. React gives you a helper type for this: `React.ComponentPropsWithoutRef<'button'>` — "every prop a normal `<button>` accepts" (without the `ref`, which we handle separately).

Then you **spread the rest** onto the real element with `...rest`, and you **merge** `className` so callers can add classes without wiping out yours.

```
import { ComponentPropsWithoutRef } from 'react';

interface ButtonProps extends ComponentPropsWithoutRef<'button'> {
  variant?: 'primary' | 'secondary';
}

function Button({ variant = 'primary', className, ...rest }: ButtonProps) {
  const classes = `btn btn-${variant} ${className ?? ''}`.trim();
  return <button className={classes} { ...rest} />;
}
```

What is happening:

- `extends ComponentPropsWithoutRef<'button'>` → our `Button` now accepts `onClick`, `disabled`, `type`, everything.
- We pull out the props we care about (`variant`, `className`) and gather everything else into `...rest` (rest = "all the other props I didn't name").
- `{ ...rest}` forwards them straight to the real `<button>` — this is called **spreading**.
- `className ?? ''` merges the caller's classes with ours (`??` = "use the left side unless it's null/undefined, then use the right"). So `<Button className="wide" disabled>Save</Button>` just works.

There is also `ComponentPropsWithRef<'button'>` when you *do* want the `ref` included in the type. For inputs we handle `ref` a smarter way — see next.

4. `forwardRef` — so an Input can be focused and used by forms

A `ref` is a **direct handle to the real DOM element** (ref = "reference to the actual `<input>` in the page"). Code sometimes needs it — to call `.focus()`, or so **React Hook Form** (Day 6) can register the field.

Problem: by default a `ref` you put on your `<Input>` component points at the component, **not** at the real `<input>` inside it. So `.focus()` would fail. The fix is `forwardRef` — it **passes the ref through** to the inner element.

Any input in a component library **must** forward its ref, or React Hook Form and `focus()` break.

```
import { ComponentPropsWithoutRef, forwardRef } from 'react';

interface InputProps extends ComponentPropsWithoutRef<'input'> {
  label?: string;
}

const Input = forwardRef<HTMLInputElement, InputProps>(
  ({ label, id, ...rest }, ref) => (
    <div className="field">
      {label} && <label htmlFor={id}>{label}</label>
      <input id={id} ref={ref} { ... rest} />
    </div>
  )
);

Input.displayName = 'Input';
```

Reading the types: `forwardRef<HTMLInputElement, InputProps>` says "the ref points to a real `HTMLInputElement`, and the props are `InputProps`." The function now gets a **second argument**, `ref`, which we attach to the inner `<input>`. Now `register('name')` from React Hook Form and manual `.focus()` both work.

React 19 note: in the newest React you can accept `ref` as a normal prop and skip `forwardRef`. But `forwardRef` still works everywhere and is what you'll see in most codebases and tutorials, so learn it first.

5. Generic components: a `Table<T>` that works for *any* row type

This is the day TypeScript **generics** really pay off. A generic is a **type placeholder** — the same idea as C# `List<T>` or `Repository<T>`. You write the code once, and the caller decides what `T` is.

A table is the perfect example. It shouldn't only know about employees — it should render **any** list of objects. So we make it generic over the row type `T`.

```

import { ReactNode } from 'react';

interface Column<T> {
  header: string;
  render: (row: T) => ReactNode;    // how to draw one cell for this column
}

interface TableProps<T> {
  rows: T[];
  columns: Column<T>[];
  rowKey: (row: T) => string | number;    // stable key per row
}

function Table<T>({ rows, columns, rowKey }: TableProps<T>) {
  return (
    <table>
      <thead>
        <tr>{columns.map((c) => <th key={c.header}>{c.header}</th>)}</tr>
      </thead>
      <tbody>
        {rows.map((row) => (
          <tr key={rowKey(row)}>
            {columns.map((c) => <td key={c.header}>{c.render(row)}</td>)}
          </tr>
        ))}
      </tbody>
    </table>
  );
}

```

Now reuse our **Employee** type from earlier days:

```

interface Employee {
  id: number;
  name: string;
  department: string;
  salary: number;
  active: boolean;
}

<Table<Employee>
  rows={employees}
  rowKey={(e) => e.id}
  columns={[
    { header: 'Name',      render: (e) => e.name },
    { header: 'Department', render: (e) => e.department },
    { header: 'Salary',    render: (e) => `$$${e.salary.toLocaleString()}` },
  ]}
/>

```

Why this is beautiful: because we wrote `Table<T>`, when you say `Table<Employee>`, TypeScript **knows** every `render: (e) => ...` gives you an `Employee`. Type `e.name` and it's a red squiggle instantly. Try to return a raw object it can't draw, error. One table component, fully type-safe for employees, departments, paylips — anything.

One tiny gotcha in `.tsx` files: for an **arrow-function** generic, write `const Table = <T,>(props) => ...` with a trailing comma — `<T,>`. Without the comma, the compiler thinks `<T>` is a JSX tag. Our example used a **function** declaration, which doesn't need the comma.

6. Composition (children) vs configuration (many props) — prefer composition

You often have two ways to make a component flexible:

- **Configuration** — add more and more props (`title`, `subtitle`, `imageUrl`, `footerText`, `showButton` ...). This grows forever and never fits every case.
- **Composition** — let the caller pass whatever markup they want as `children`. The component just provides the frame.

Prefer composition. It is simpler and endlessly flexible. Think of a `Card` as a picture frame: the frame is fixed, but you can put *any* picture inside.


```
import { ReactNode } from 'react';

interface CardProps {
  title?: string;
  children: ReactNode; // put anything you like inside
}

function Card({ title, children }: CardProps) {
  return (
    <section className="card">
      {title} && <header className="card-title">{title}</header>
      <div className="card-body">{children}</div>
    </section>
  );
}
```

```
<Card title="Employee of the Month">
  
  <p>Jane Doe - Engineering</p>
  <Button>View profile</Button>
</Card>
```

The `Card` has no idea what's inside — and that's why it works for a profile, a chart, or a form. Reach for props when you need to *configure behavior* (a `variant`, an `onClick`); reach for `children` when you need to *fill a slot* with content.

7. Modal + Portals: escaping the layout trap

A modal is a pop-up dialog over the page. There's a classic bug: if the modal lives deep inside a container that has `overflow: hidden` or a tricky `z-index` (z-index = stacking order, who sits on top), the modal gets clipped or hidden behind other things.

The fix is a **portal**. `createPortal` (from `react-dom`) renders your JSX into a **different place in the real DOM** — usually straight into `document.body` — while it still behaves like a normal child in your React tree. So the modal escapes all those parent traps and sits on top of everything.

```

import { useEffect, ReactNode } from 'react';
import { createPortal } from 'react-dom';

interface ModalProps {
  isOpen: boolean;
  onClose: () => void;
  title?: string;
  children: ReactNode;
}

function Modal({ isOpen, onClose, title, children }: ModalProps) {
  // Close when the user presses Escape
  useEffect(() => {
    if (!isOpen) return;
    const onKey = (e: KeyboardEvent) => {
      if (e.key === 'Escape') onClose();
    };
    document.addEventListener('keydown', onKey);
    return () => document.removeEventListener('keydown', onKey); // cleanup
  }, [isOpen, onClose]);

  if (!isOpen) return null;

  return createPortal(
    <div className="overlay" onClick={onClose}>
      {/* stopPropagation = "don't let a click inside bubble up and close it" */}
      <div
        className="modal"
        role="dialog"
        aria-modal="true"
        onClick={(e) => e.stopPropagation()}
      >
        {title && <h2>{title}</h2>}
        {children}
      </div>
    </div>,
    document.body // ← render into body, not where we sit in the tree
  );
}

```

Notice the modal is **presentational**: it doesn't decide *when* it's open — the parent owns `isOpen` and passes `onClose`. Two closing paths: press **Escape** (the effect) or click the dark **overlay** (background). Clicking inside the box calls `stopPropagation` so it does *not* close.

A word on **accessibility** (accessibility = making it usable for keyboard and screen-reader users):

`role="dialog"` and `aria-modal="true"` tell assistive tech "this is a dialog." Ideally you'd also move keyboard focus into the modal when it opens and trap it there. A small, well-tested library (like `@headlessui/react` or `react-focus-lock`) usually handles focus-trapping for you — you don't have to hand-roll it.

8. Loader and Pagination — plain prop-driven components

Not everything is fancy. Most of a UI kit is small, boring, prop-driven pieces. That's a good thing.

Loader — shows a spinner/message while data loads:

```
interface LoaderProps {
  label?: string;
}

function Loader({ label = 'Loading...' }: LoaderProps) {
  return <div className="loader" role="status">{label}</div>;
}
```

Pagination — shows page controls and *reports* clicks; it does **not** own the page number (the parent does):

```

interface PaginationProps {
  page: number;
  pageCount: number;
  onPageChange: (page: number) => void;
}

function Pagination({ page, pageCount, onPageChange }: PaginationProps) {
  return (
    <div className="pagination">
      <button disabled={page ≤ 1} onClick={() => onPageChange(page - 1)}>
        Prev
      </button>
      <span>Page {page} of {pageCount}</span>
      <button disabled={page ≥ pageCount} onClick={() => onPageChange(page + 1)}>
        Next
      </button>
    </div>
  );
}

```

See the pattern again: it takes **data** (`page` , `pageCount`) and a **callback** (`onPageChange`). It knows nothing about employees. That's what makes it reusable.

9. Custom hooks: reusable *logic*

A **custom hook** is just a **function whose name starts with** `use` and that calls other hooks (`useState` , `useEffect` , ...) inside it. That's the whole definition. In C# terms: you noticed the same few lines repeated in several components, so you extracted them into a shared helper — except this helper is allowed to use React state.

Why bother? Two reasons: **reuse the stateful logic** in many components, and **keep your components clean** (the component reads like a summary; the messy details live in the hook).

`useToggle` — a true/false switch (open/closed, on/off). We return a **tuple** (a fixed-length pair) just like `useState` does:

```
import { useState } from 'react';

function useToggle(initial = false): [boolean, () => void] {
  const [value, setValue] = useState(initial);
  const toggle = () => setValue((v) => !v);
  return [value, toggle]; // [current value, function to flip it]
}

// In a component:
const [isOpen, toggleOpen] = useToggle();
// <Button onClick={toggleOpen}>Open</Button>
// <Modal isOpen={isOpen} onClose={toggleOpen}> ... </Modal>
```

useDebounce — "wait until the user stops typing." Great for a search box so you don't fire a request on every keystroke. It's **generic** (`<T>`) so it debounces any value. Debounce = "delay a value until things go quiet for a moment."

```
import { useState, useEffect } from 'react';

function useDebounce<T>(value: T, delay = 300): T {
  const [debounced, setDebounced] = useState(value);

  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id); // cancel if value changes first
  }, [value, delay]);

  return debounced;
}

// const debouncedSearch = useDebounce(searchText, 400);
// then only fetch when debouncedSearch changes
```

useEmployees — a **data hook** that wraps Day 5's fetch + loading + error logic and hands back a clean object. Now every screen that needs employees just calls `useEmployees()` — no repeated fetch boilerplate.

```

import { useState, useEffect } from 'react';

interface Employee {
  id: number;
  name: string;
  department: string;
  salary: number;
  active: boolean;
}

interface UseEmployeesResult {
  data: Employee[];
  loading: boolean;
  error: string | null;
}

function useEmployees(): UseEmployeesResult {
  const [data, setData] = useState<Employee[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    let active = true;    // guard against setting state after unmount
    setLoading(true);

    fetch('/api/employees')
      .then((res) => {
        if (!res.ok) throw new Error('Request failed');
        return res.json() as Promise<Employee[]>;
      })
      .then((json) => {
        if (active) { setData(json); setError(null); }
      })
      .catch((err: unknown) => {
        if (active) setError(err instanceof Error ? err.message : 'Unknown error');
      })
      .finally(() => {
        if (active) setLoading(false);
      });

    return () => { active = false; };    // cleanup on unmount
  });
}

```

```

}, []);

return { data, loading, error };
}

```

And now the component that uses **everything** — the hook returns clean data, and the UI kit renders it:

```

function EmployeeList() {
  const { data, loading, error } = useEmployees();

  if (loading) return <Loader />;
  if (error) return <p role="alert">{error}</p>;

  return (
    <Table<Employee>
      rows={data}
      rowKey={(e) => e.id}
      columns={[
        { header: 'Name', render: (e) => e.name },
        { header: 'Salary', render: (e) => `$$${e.salary.toLocaleString()}` },
      ]}
    />
  );
}

```

Look how short and readable that component is. All the fetch mess is hidden in the hook; all the table mess is hidden in the component. That's the payoff.

Tip: return an **object** (`{ data, loading, error }`) from data hooks — the caller can name and pick what it wants. Return a **tuple** (`[value, toggle]`) only for tiny 2-item hooks like `useToggle` , mirroring `useState` .

10. Rules of Hooks apply inside custom hooks too

Custom hooks are still hooks, so the same two rules from Day 4 apply — inside your hook as well:

1. **Only call hooks at the top level.** Never inside an `if` , a loop, or a nested function. React relies on hooks being called in the **same order** every render.
2. **Only call hooks from React functions** — a component, or another custom hook. Not from a plain utility function or an event handler.

And the naming rule is real, not just style: the name **must start with use**. That's how React's linter knows to check the rules for it. `useToggle` = checked. `toggleThing` = React won't treat it as a hook, and you'll get subtle bugs.

```
// ❌ Wrong – hook called conditionally (this would be a bug)
// function useThing(flag: boolean) {
//   if (flag) { const [x] = useState(0); } // never do this
// }

// ✅ Right – hook at the top, branch on the value afterwards
function useThing(flag: boolean) {
  const [x, setX] = useState(0);
  return flag ? x : 0;
}
```

11. Organizing the library, and testing in isolation

Keep the UI kit together in one folder, e.g. `src/ui/`, one file per component (`Button.tsx`, `Input.tsx`, `Table.tsx`, ...). Put the shared hooks in `src/hooks/`.

Add a **barrel file** — an `index.ts` that re-exports everything — so callers import from one tidy path:

```
// src/ui/index.ts (a "barrel")
export { Button } from './Button';
export { Input } from './Input';
export { Table } from './Table';
export { Modal } from './Modal';
export { Loader } from './Loader';
export { Pagination } from './Pagination';
```

```
// Now anywhere in the app:
import { Button, Modal, Table } from '../ui';
```

Because these components are prop-driven, you can **test them in isolation** — render one on its own with sample props, no app or backend needed. The popular tool for this is **Storybook**: it shows each component alone in a little sandbox, in all its states (primary button, disabled button, loading table). It's a living catalog of your UI kit and a fast way to eyeball every variation. (Setting up Storybook is optional and comes later — just know the phrase and why it exists.)

One-line summary

Write UI once as small, typed, prop-driven components (generic where the data varies, like `Table<T>`), pull shared logic into `use*` custom hooks, and your HR app becomes short, consistent, and safe — reuse instead of copy-paste.

3 Common Mistakes to Avoid

A reusable component is like a shared control in your old world — a WinForms/Blazor user control, or a helper class everyone reuses. A custom hook is just shared logic pulled out into a helper (`use*` function), the same way you'd move common code into a service in C#. Here are the traps beginners fall into on Day 7, and the easy fix for each.

1. Hardcoding text or values inside a component

If the words live inside the component, you can only ever use it one way. That is not reusable.

❌ **Wrong** — the label is baked in:

```
function SaveButton() {  
  return <button>Save Employee</button>; // stuck saying this forever  
}
```

✅ **Right** — pass it in as a prop (a prop is just an input, like a method parameter):

```
function Button({ children }: { children: React.ReactNode }) {  
  return <button>{children}</button>;  
}  
  
// <Button>Save Employee</Button> / <Button>Delete</Button>
```

Rule: if it changes from screen to screen, it should be a prop.

2. Not spreading `...rest` and not forwarding `className`

If you only pass the props you thought of, the component can't accept native HTML attributes (`type` , `disabled` , `aria-label`) and can't be styled from outside. Extend the real element's props and spread the leftovers.

❌ **Wrong** — `type` , `disabled` , `className` all get dropped:

```
function Button({ children }: { children: React.ReactNode }) {  
  return <button>{children}</button>;  
}
```

✅ **Right** — take everything a real `<button>` takes, then pass it through:

```
import { ComponentPropsWithoutRef } from 'react';

interface ButtonProps extends ComponentPropsWithoutRef<'button'> {
  variant?: 'primary' | 'secondary';
}

function Button({ variant = 'primary', className, ...rest }: ButtonProps) {
  return (
    <button className={`btn btn-${variant} ${className ?? ''}`} {...rest} />
  );
}
```

Now `<Button type="submit" disabled aria-label="Save" className="mt-2">` all just work.
`ComponentPropsWithoutRef<'button'>` = "give me every prop a normal button accepts."

3. Forgetting `forwardRef` on an Input

A ref is a direct handle to the real DOM element (for `.focus()`, or for React Hook Form's `register`). A normal function component **cannot** receive a ref — it gets lost. Use `forwardRef` so the ref reaches the actual `<input>`.

✗ Wrong — the ref never lands on anything, focus/register break:

```
function Input(props: { label: string }) {
  return <input />; // ref passed by the parent is silently ignored
}
```

✓ Right — forward the ref to the real input:

```
import { forwardRef, ComponentPropsWithoutRef } from 'react';

interface InputProps extends ComponentPropsWithoutRef<'input'> {
  label: string;
}

const Input = forwardRef<HTMLInputElement, InputProps>(
  ({ label, id, ...rest }, ref) => (
    <label htmlFor={id}>
      {label}
      <input id={id} ref={ref} { ... rest} />
    </label>
  )
);

Input.displayName = 'Input';
```

Now `ref={ ... }` and RHF's `{ ... register('name')}` reach the input correctly.

4. Not typing `children`

`children` is whatever you put between the tags. Don't leave it as `any` — type it as `React.ReactNode`, which means "anything React can render" (text, numbers, elements, a list of them, or nothing).

✗ Wrong — untyped, no safety, no autocomplete:

```
function Card({ title, children }: { title: string; children: any }) {
  return <section><h3>{title}</h3>{children}</section>;
}
```

✓ Right:

```
import { ReactNode } from 'react';

interface CardProps {
  title: string;
  children: ReactNode;
}

function Card({ title, children }: CardProps) {
  return (
    <section>
      <h3>{title}</h3>
      {children}
    </section>
  );
}
```

5. Building a Modal without `createPortal`

If a modal is rendered deep inside the tree, a parent's `overflow: hidden` or a low `z-index` can clip or hide it. A **portal** renders the modal into `document.body` instead, so it floats above everything — while still behaving like a normal child in your code.

✗ Wrong — modal lives inside the card, gets clipped:

```
function Modal({ children }: { children: React.ReactNode }) {
  return <div className="modal">{children}</div>; // trapped in parent overflow
}
```

✓ Right — send it to `document.body` with a portal:

```
import { ReactNode } from 'react';
import { createPortal } from 'react-dom';

function Modal({ children }: { children: React.ReactNode }) {
  return createPortal(
    <div className="modal">{children}</div>,
    document.body
  );
}
```

6. A Modal that won't close on Escape / overlay — or leaks its key listener

Users expect Escape and a click on the dark backdrop to close the dialog. And any listener you add in an effect **must** be removed in the cleanup, or you leak listeners every time the modal opens.

❌ **Wrong** — no Escape, no cleanup (listener added again and again):

```
useEffect(() => {  
  document.addEventListener('keydown', onKey); // never removed → leak  
});
```

✅ **Right** — Escape + overlay close, with cleanup:

```

import { useEffect, ReactNode } from 'react';
import { createPortal } from 'react-dom';

interface ModalProps {
  open: boolean;
  onClose: () => void;
  children: ReactNode;
}

function Modal({ open, onClose, children }: ModalProps) {
  useEffect(() => {
    if (!open) return;
    const onKey = (e: KeyboardEvent) => {
      if (e.key === 'Escape') onClose();
    };
    document.addEventListener('keydown', onKey);
    return () => document.removeEventListener('keydown', onKey); // cleanup!
  }, [open, onClose]);

  if (!open) return null;

  return createPortal(
    <div className="overlay" onClick={onClose}>
      {/* stop the click inside the box from bubbling up and closing it */}
      <div className="modal" onClick={e => e.stopPropagation()}>
        {children}
      </div>
    </div>,
    document.body
  );
}

```

The `return () => ...` inside `useEffect` is the cleanup — think of it like `Dispose()`: React runs it to tidy up before re-running or unmounting.

7. Custom hook not named `use*`

The `use` prefix is not decoration — the React lint rules use the name to know it's a hook and check the Rules of Hooks. Name it anything else and the tooling stops protecting you.

✗ Wrong:

```
function toggleValue(initial = false) {    // lint won't treat this as a hook
  const [on, setOn] = useState(initial);
  return [on, () => setOn((v) => !v)] as const;
}
```

✅ **Right** — start the name with `use` :

```
import { useState, useCallback } from 'react';

function useToggle(initial = false) {
  const [on, setOn] = useState(initial);
  const toggle = useCallback(() => setOn((v) => !v), []);
  return [on, toggle] as const;
}
```

(`as const` makes the return a fixed `[boolean, () => void]` tuple, so you can destructure it like `useState` .)

8. Calling hooks conditionally inside a custom hook

Rules of Hooks: hooks must run in the same order on every render — so always call them at the top level, never inside an `if`, loop, or after an early `return`. Put the condition *inside* the hook instead.

❌ **Wrong** — the hook is skipped when `id` is missing:

```
// function useUserData(id?: string) {
//   if (!id) return null;           // early return ...
//   const [data, setData] = useState(null);    // ...so this sometimes runs, sometimes not
//   - breaks Rules of Hooks
// }
```

✅ **Right** — call the hooks always; branch inside the effect:


```
import { useState, useEffect } from 'react';

function useUserData(id?: string) {
  const [data, setData] = useState<string | null>(null);

  useEffect(() => {
    if (!id) return;          // condition lives INSIDE the hook
    setData(`user-${id}`);
  }, [id]);

  return data;
}
```

9. Forgetting a stable `key` when the Table renders rows

React needs a stable, unique `key` per row to track which is which. Don't use the array index — it changes when rows are added, removed, or sorted, which causes wrong rows to update. Use a real id from the data.

❌ **Wrong** — index as key:

```
{rows.map((row, i) => <tr key={i}> ... </tr>)} // breaks on sort/insert/delete
```

✅ **Right** — let the caller say what's unique, using a generic `Table<T>`:

```

import { ReactNode } from 'react';

interface Column<T> {
  header: string;
  render: (row: T) => ReactNode;
}

interface TableProps<T> {
  rows: T[];
  columns: Column<T>[];
  rowKey: (row: T) => string | number; // caller picks the stable id
}

function Table<T>({ rows, columns, rowKey }: TableProps<T>) {
  return (
    <table>
      <thead>
        <tr>{columns.map((c) => <th key={c.header}>{c.header}</th>)}</tr>
      </thead>
      <tbody>
        {rows.map((row) => (
          <tr key={rowKey(row)}>
            {columns.map((c) => <td key={c.header}>{c.render(row)}</td>)}
          </tr>
        ))}
      </tbody>
    </table>
  );
}

// <Table rows={employees} columns={cols} rowKey={(e) => e.id} />

```

The `<T>` makes the Table generic — like `List<T>` in C#. One Table works for employees, departments, anything, and stays fully typed.

10. Over-engineering — a component with 20 props

Trying to cover every case with more and more props (`showFooter`, `footerText`, `iconPosition`, `badgeColor` ...) makes the component painful to use and remember. Prefer **composition** — let the caller pass content via `children`.

✗ Wrong — too many props, rigid, hard to read:

```
// <Card title="Employees" subtitle="Team" showFooter footerText="Save"
//       icon="user" iconPosition="left" badge="3" badgeColor="red" />
```

✓ **Right** — a simple shell, fill it with children:

```
// <Card>
//   <h3>Employees</h3>
//   <EmployeeTable />
//   <Button>Save</Button>
// </Card>
```

Rule of thumb: don't add a prop until you actually have a second, different use for it. Build for what you need now, not every imagined future.

11. Duplicating fetch/effect logic across many components

Copy-pasting the same `fetch` + `useState` + `useEffect` into every screen is like rewriting the same DB call in five services. Pull it into **one custom hook** and reuse it everywhere.

✗ **Wrong** — the same loading/fetch block copied into Dashboard, List, Report...

✓ **Right** — one hook owns the logic; every component just calls it:

```
import { useState, useEffect } from 'react';

interface Employee {
  id: number;
  name: string;
}

function useEmployees() {
  const [data, setData] = useState<Employee[]>([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    let active = true; // guard so we don't set state after unmount
    fetch('/api/employees')
      .then((res) => res.json() as Promise<Employee[]>)
      .then((json) => { if (active) setData(json); })
      .finally(() => { if (active) setLoading(false); });
    return () => { active = false; };
  }, []);

  return { data, loading };
}

// const { data, loading } = useEmployees(); // in any component
```

Fix the fetch once, and every screen gets the fix. That's the whole point of a custom hook.

Heads-up: global state (Context + reducers) and performance tuning (`memo` , `useMemo` , `useCallback`) are **Day 8**. For today, keep every component presentational and prop-driven — data in through props, events out through callbacks.

4 Concept Notes — Recall & Interview (copy by pen)

Quick concept recall for Day 7 — reusable components and custom hooks. Read each **Definition**, then the pros/cons. The 💡 lines are the classic interview answers — memorise those. Code is tiny inline only, so it's easy to copy by pen.

Reusable Components & Composition

Reusable Component

Definition:

- A small, self-contained piece of UI you build once and use in many places by passing different props.
- C#/.NET: like a WinForms/Blazor user control or a shared class — write once, reuse everywhere.

✅ Advantages:

- Write once, use everywhere — less duplicate code.
- One place to fix a bug or change the style.
- Keeps the look and feel consistent across the app.

❌ Disadvantages / watch-outs:

- Making it "do everything" adds too many props and gets messy.
- Going generic too early wastes effort — build it when you actually repeat code.

💡 Interview tip:

- A good reusable component is "prop-driven and presentational" — data goes in, UI comes out, no business logic inside.

📄 Example:

- `<Button variant="primary" onClick={save}>Save</Button>` — same button, different props.

Presentational vs Container Components

Definition:

- Presentational = "how it looks" (props → UI). Container = "how it works" (fetches data, holds state, passes props down).
- C#/.NET: presentational ~ a dumb view/user control; container ~ the code-behind or controller that feeds it.

✅ Advantages:

- Clear split of jobs — easy to test and reuse the presentational parts.
- UI-kit components stay presentational, so any page can reuse them.

❌ Disadvantages / watch-outs:

- The old strict pattern is less common now — hooks let one component do both.
- Don't over-split tiny components just to follow the rule.

💡 **Interview tip:**

- Presentational = no data fetching, driven by props; container = owns state/data and renders presentational children. Add that hooks blurred this line.

📄 **Example:**

- `<EmployeeList items={data} />` (presentational) fed by a container that loaded `data`.

Props API Design

Definition:

- Deciding a component's "public interface" — which props it takes, their names, types, and defaults.
- C#/.NET: like designing a class's public constructor/properties — the contract callers use.

✅ **Advantages:**

- Good prop names make a component usable without reading its code.
- Small, focused prop lists are easier to remember and type-check.

❌ **Disadvantages / watch-outs:**

- Too many boolean props (`isPrimary` , `isBig` , `isDanger`) → use one union like `variant`.
- Don't leak internal details as props; keep the surface small.

📄 **Example:**

- `variant?: 'primary' | 'secondary' | 'danger'` — one clear prop beats three booleans.

Children & Composition (composition over configuration)

Definition:

- `children` is whatever you put between a component's tags; composition means building UI by nesting components instead of adding endless props.
- C#/.NET: like a Blazor `RenderFragment` / `ChildContent` , or a Razor layout `@Body`.

✅ **Advantages:**

- Flexible — the parent decides what goes inside without new props.
- Keeps components simple; you compose small parts into bigger ones.

❌ **Disadvantages / watch-outs:**

- Deep nesting can get hard to read.
- Passing many "slots" as props (render props) can get verbose.

💡 **Interview tip:**

- "Composition over inheritance": React reuses code by nesting/passing components (and `children`), not by extending classes.

Example:

- `<Card><h3>Title</h3><p>Body</p></Card>` — `Card` just renders `{children}`.

Default Props

Definition:

- Values a prop falls back to when the caller doesn't pass it.
- In function components, set them with default parameter values in destructuring — not the old `Component.defaultProps` (deprecated for function components).

✓ Advantages:

- Callers only pass what they need; sensible defaults fill the rest.
- Fewer `undefined` bugs.

✗ Disadvantages / watch-outs:

- `defaultProps` on function components is deprecated — prefer default values.
- A default object/array is a new reference every render (matters for memo/deps — Day 8).

Example:

- `function Button({ variant = 'primary' }: Props)` — default set right in the parameter.

Typing, Refs & Generics

Extending Native Element Props (ComponentPropsWithoutRef)

Definition:

- A TS helper that gives your component all the real props of an HTML element (like `<button>`), so it accepts them too.
- C#/.NET: like inheriting a base control's properties so you don't redeclare `onClick`, `disabled`, etc.

✓ Advantages:

- Your `Button` supports `onClick`, `type`, `disabled`, `aria-*` for free — fully typed.
- Spread the rest with `{ ...rest }` so no native prop is lost.

✗ Disadvantages / watch-outs:

- Watch for clashes between your custom prop and a native one.
- `WithoutRef` drops `ref`; use `ComponentPropsWithRef` (or `forwardRef`) if you need the ref.

Example:

- `interface Props extends React.ComponentPropsWithoutRef<'button'> { variant?: string }` — button props + your extras.

forwardRef

Definition:

- Lets a parent pass a `ref` through your component to a real DOM node inside it (like the actual `<input>`).
- C#/.NET: a bit like exposing an inner control so the parent can call `.Focus()` on it.

✓ Advantages:

- Parents can focus, scroll to, or measure the inner element.
- Needed for reusable Input components and form libraries.

✗ Disadvantages / watch-outs:

- Without it, a `ref` on your function component does nothing.
- React 19: you can take `ref` as a normal prop, so `forwardRef` is becoming optional.

💡 Interview tip:

- Why forwardRef? Function components don't receive `ref` as a prop by default; `forwardRef` forwards it down to a child DOM node (e.g. to focus an input).

📄 Example:

- `forwardRef<HTMLInputElement, Props>((props, ref) => <input ref={ref} { ... props} />)` .

Generic Components (Table<T>)

Definition:

- A component with a type parameter `<T>` so it works with any row shape while staying type-safe.
- C#/.NET: exactly like a generic class/method — `List<T>` or `Grid<T>` .

✓ Advantages:

- One `Table` works for employees, departments, anything — no `any` .
- Column config and row callbacks are typed to the real data.

✗ Disadvantages / watch-outs:

- Syntax is fiddly in `.tsx` — use the `<T,>` comma trick so TS doesn't read it as JSX.
- Don't over-generalise; go generic only when you truly reuse it.

💡 Interview tip:

- Type it as a generic function component: `function Table<T>(props: TableProps<T>)` , with a `columns` array and `keyField` typed against `T` .

📄 Example:

- `<Table<Employee> rows={emps} columns={cols} />` — rows and columns typed as `Employee` .

Controlled vs Uncontrolled (library components)

Definition:

- Controlled = the parent owns the value via `value` + `onChange` . Uncontrolled = the component keeps its own state (parent reads it via a ref).

- C#/.NET: controlled ~ two-way binding you drive; uncontrolled ~ the control manages its own state.

✅ Advantages:

- Controlled: parent has full control and one source of truth.
- Uncontrolled: less wiring for simple cases; supports a `defaultValue`.

❌ Disadvantages / watch-outs:

- Never mix both — passing `value` without `onChange` makes React warn about a read-only input.
- For a reusable input, pick one mode and document it clearly.

📄 Example:

- `<Input value={name} onChange={e => setName(e.target.value)} />` — controlled.

Portals

Portals (createPortal)

Definition:

- Renders a component's DOM somewhere else in the page (like `<body>`) while it still behaves like a normal child in React.
- Perfect for modals, tooltips, and dropdowns that must sit above everything.

✅ Advantages:

- Escapes parent `overflow: hidden` / `z-index` traps — the modal shows on top.
- Events still bubble through the React tree, not the DOM position.

❌ Disadvantages / watch-outs:

- You need a target DOM node to render into (e.g. a `#modal-root` div or `document.body`).
- Remember accessibility: focus trap, `Esc` to close, and lock background scroll.

💡 Interview tip:

- What's a portal? `createPortal(children, domNode)` renders children into a different DOM node but keeps them in the React tree, so events bubble as if nested.

📄 Example:

- `createPortal(<Modal/>, document.body)` — renders the modal at `<body>`.

Custom Hooks & Library Structure

Custom Hook (what & why)

Definition:

- A plain function whose name starts with `use` that calls other hooks to share stateful logic between components.
- C#/.NET: like pulling shared logic into a helper/service so many callers reuse it.

✅ Advantages:

- Reuse logic (fetching, toggling, debouncing) without copy-paste.
- Keeps components small; the logic is testable on its own.
- Each component that uses the hook gets its OWN separate state.

❌ Disadvantages / watch-outs:

- The name must start with `use` or React can't apply the Rules of Hooks.
- A hook shares logic, not state — two components don't share the same values.

💡 Interview tip:

- What is a custom hook and why? A `use*` function that extracts and reuses stateful logic; unlike a component it returns data/handlers, not JSX.

📄 Example:

- `const [open, toggle] = useToggle(false)` — reusable open/close logic.

Rules of Hooks

Definition:

- Only call hooks at the top level of a component or another hook — never inside loops, conditions, or nested functions — and only from React functions.

✅ Advantages:

- Keeps the hook call order the same every render, so React tracks state correctly.
- The `eslint-plugin-react-hooks` rule catches breaks automatically.

❌ Disadvantages / watch-outs:

- Calling a hook inside an `if` or loop breaks state — a classic bug.
- Put the condition *inside* the hook, not the hook inside the condition.

💡 Interview tip:

- Two rules: (1) call hooks only at the top level, (2) call them only from React functions/custom hooks — so call order stays stable each render.

📄 Example:

- `if (x) useState()` ❌ — never call a hook conditionally.

Sharing Stateful Logic with Hooks

Definition:

- Move repeated `useState` / `useEffect` logic into a custom hook and return the values + functions components need.
- C#/.NET: like moving duplicate code into a shared service method that returns a result.

✅ Advantages:

- Replaces older patterns (HOCs, render props) with simpler functions.
- One tested hook (e.g. `useEmployees`) can feed many pages.

✗ Disadvantages / watch-outs:

- Return a clear shape — an array like `useState`, or an object when there are many values.
- Don't hide too much; the caller should still understand what it gets back.

Example:

- `const { data, loading, error } = useFetch<Employee[]>('/api/employees')` — shared fetch logic.

Barrel Export / Library Structure

Definition:

- A single `index.ts` that re-exports all your components so others import them from one place.
- C#/.NET: like a shared class library / namespace you reference once.

✓ Advantages:

- Clean imports: `import { Button, Modal } from '@/ui'` instead of many long paths.
- The kit's public API is visible in one file.

✗ Disadvantages / watch-outs:

- Big barrels can hurt tree-shaking / bundle size if you're not careful.
- A barrel that imports itself can cause circular-import issues.

Example:

- `export { Button } from './Button'` — one line per component in `ui/index.ts`.